GitHub Copilot in the CLI

Plan, implement, review in your terminal

Session M2-01 | 3 hours

Agenda

Segment	Time
What the CLI is, install, sign in	8 min
Interactive vs `copilot -p`	7 min
Plan mode and the core loop	10 min
Slash commands that matter	8 min
Permissions and context	12 min
Resume, CLI vs IDE, advanced	10 min
Lab handoff	5 min

What the CLI is

`copilot` is an **agent in your terminal**. It reads files, edits code, and runs commands. It asks before risky actions.

```
npm install -g @github/copilot
cd my-project
copilot          # first run: sign in, trust the folder
```

- Needs Node.js 22 or later and a Copilot plan.
- Also installs with Homebrew and WinGet.
- **Trust the folder** only if you own it.

Interactive or programmatic

	Interactive	Programmatic
Start	`copilot`	`copilot -p "prompt"`
Shape	Conversation	One answer, then exit
Permissions	You answer prompts	You allow tools up front
Use for	Real work	Scripts, quick questions, CI

```
copilot -p "Summarize the last commit" --allow-tool='shell(git log)'
```

The core loop: plan, implement, review

1. **Plan.** `Shift+Tab` or `/plan`. Copilot proposes, edits nothing.
2. **Read and revise the plan.** Cheapest place to fix a mistake.
3. **Implement.** Approve the plan and the tools.
4. **Review.** `/diff`, then `/review`. Run the tests yourself.

Skip planning for a one-line fix. Use it for anything that touches several files.

Slash commands that matter

Command	Does
`/help`	Lists commands
`/model`	Picks the model
`/context`	Shows context window use
`/compact`	Summarizes history to free space
`/diff`	Shows changes made this session
`/review`	A separate agent reviews changes
`/resume`	Picks an older session
`/skills`, `/mcp`	Shows skills and MCP servers

Permissions

Copilot asks before tools that can change things. Prompt labels vary by version.

Answer	When
Allow once	Default. You are still learning what runs.
Allow for session	Commands you trust, like `npm test`.
Deny	Anything you don't understand or need.

- Flags: `--allow-tool`, `--deny-tool`. Deny wins over allow.
- **Avoid** `/allow-all` and `/yolo`. They remove every prompt.
- Read the command before you approve it.

Context

Copilot only knows what is in its **context window**.

- `@file` attaches a file to one prompt.
- `.github/copilot-instructions.md` loads in every session. Put commands and rules there.
- `/context` shows what is used. `/compact` frees space.

Good instruction: `Run npm test after each change. Don't touch tests.`
Weak instruction: `Write good code.`

Resume a session

```
copilot --continue     # reopen the latest session
```

`/resume` opens a picker for older sessions.

- A resumed session keeps its history. It also keeps its **old context**.
- Start fresh with `/clear` or `/new` when the task changes.

CLI or IDE?

CLI wins	IDE wins
SSH and remote machines	Inline suggestions as you type
Scripts and CI (`copilot -p`)	Debugger, breakpoints
Multi-step tasks you review once	Rich visual diffs
Editor-independent teams	Selecting code to ask about

Many people use both. Start the task where you can see the work best.

Pros and cons

Pros	Cons
Works anywhere a terminal does	No inline completions
Scriptable with `-p`	Diffs are text only
Plan mode before edits	Permission prompts take attention
Same repo instructions as the IDE	Long sessions lose focus

Tips and common mistakes

Tips

- Name files with `@`. Ask for a plan first. Run tests yourself.
- Commit before a big task so `git diff` is clean.

Mistakes

- Approving every prompt without reading.
- Using `/allow-all` to stop prompts.
- Never starting a new session.
- Accepting a plan you skimmed.

Advanced: fleet

One line: `/fleet` runs parts of a task as parallel subagents.

Tradeoff: faster on independent pieces, but it uses more requests and pieces can collide.

Use `/tasks` to watch the work.

Advanced: Autopilot

One line: Autopilot mode lets Copilot keep working without asking after each step.

Tradeoff: less babysitting, less control. Give a small goal and check the diff after.

`Shift+Tab` cycles Standard, Plan, and Autopilot.

Advanced: worktree

One line: a Git worktree is a second working folder on its own branch, sharing one repository.

Tradeoff: a safe place for a risky or parallel task, but one more folder to merge and clean up.

`/new worktree` starts a session in one.

Advanced: JSONL output

One line: `--output-format json` prints one JSON object per line for other programs to read.

Tradeoff: easy to parse, but you must find the answer among event lines.

```
copilot -p "List the routes" --output-format json
```

Lab

Fix three bugs: plan, implement, review

2 hours. Optional stretch tasks cover the advanced slides.